

Assignment 5

Scikit-Learn for Machine Learning

Q1. Scikit-Learn -- Data Preprocessing (Scaling & Encoding)

Learn to prepare raw data for machine learning models using scaling and encoding techniques.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder, OneHotEncoder

np.random.seed(42)
df = pd.DataFrame({
    'Age': [25, 30, 35, 40, 28, 32, 45, 50],
    'Salary': [50000, 60000, 75000, 90000, 55000, 65000, 100000, 120000],
    'Department': ['IT', 'HR', 'IT', 'Sales', 'HR', 'Sales', 'IT', 'Sales'],
    'Gender': ['Male', 'Female', 'Female', 'Male', 'Male', 'Female', 'Male', 'Female'],
    'Performance': ['Good', 'Average', 'Excellent', 'Good', 'Average', 'Excellent', 'Good', 'Average']
})
```

Perform the following tasks:

- Apply StandardScaler to the Age and Salary columns. Print the mean and standard deviation of the scaled data to verify it has zero mean and unit variance.
- Apply MinMaxScaler to the Age and Salary columns. Verify that all scaled values lie between 0 and 1.
- Use LabelEncoder to convert the Gender column into numeric labels (0 and 1). Display the mapping between original labels and encoded values.
- Use OneHotEncoder to convert the Department column into dummy variables. Merge the encoded columns back into the original dataframe and drop the original Department column.
- Create a new column Performance_Score by manually mapping Excellent=3, Good=2, Average=1 using the map() function.
- Combine all preprocessing steps above into a single clean dataframe where every column is numeric and ready for machine learning.

Explain in 2-3 sentences: When should you use StandardScaler versus MinMaxScaler? What happens if you forget to scale features before training a model?

Q2. Scikit-Learn -- Train-Test Split & Linear Regression

Learn to split data into training and testing sets and build your first regression model.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

np.random.seed(42)
n = 200
X = pd.DataFrame({
    'Study_Hours': np.random.uniform(1, 10, n),
    'Attendance': np.random.uniform(60, 100, n),
    'Sleep_Hours': np.random.uniform(4, 9, n)
})
y = 5 * X['Study_Hours'] + 0.3 * X['Attendance'] - 2 * X['Sleep_Hours'] + np.random.normal(0,
```

Perform the following tasks:

- Split the data into training and testing sets using an 80-20 split. Set random_state=42 for reproducibility.
- Print the shape of X_train, X_test, y_train, and y_test to verify the split.
- Create a LinearRegression model, train it on the training data, and print the learned coefficients and intercept.
- Use the trained model to make predictions on the test set. Create a scatter plot of actual versus predicted values.
- Calculate and print the Mean Squared Error (MSE) and R-squared score on the test set.
- Interpret the coefficients: which feature has the strongest positive impact on the target variable?

Explain in 2-3 sentences: Why is it important to split data into training and testing sets? What problem would occur if you evaluated the model on the same data it was trained on?

Q3. Scikit-Learn -- Logistic Regression & Classification

Build a classification model to predict categorical outcomes and evaluate its performance.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler

np.random.seed(42)
n = 300
df = pd.DataFrame({
    'Age': np.random.randint(18, 60, n),
    'Income': np.random.normal(50000, 15000, n),
    'Credit_Score': np.random.randint(300, 850, n)
})
# Create target: approved if credit score > 650 and income > 45000
df['Approved'] = ((df['Credit_Score'] > 650) & (df['Income'] > 45000)).astype(int)
```

Perform the following tasks:

- Separate the features (Age, Income, Credit_Score) from the target (Approved). Split into 75-25 train-test with random_state=42.
- Apply StandardScaler to the feature columns before training. Fit the scaler on training data only, then transform both train and test.
- Train a LogisticRegression model on the scaled training data. Set max_iter=1000.
- Make predictions on the test set and calculate the accuracy score.
- Print the confusion matrix and interpret each of the four values (TN, FP, FN, TP).
- Print the classification report showing precision, recall, and F1-score for each class.

Explain in 2-3 sentences: What is the difference between precision and recall? In what scenario would you prioritize one over the other?

Q4. Scikit-Learn -- K-Means Clustering (Unsupervised Learning)

Discover hidden groups in data without labeled targets. Clustering is a powerful EDA technique.

```
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

np.random.seed(42)
# Create 3 distinct clusters
cluster1 = np.random.normal(loc=[20, 30], scale=3, size=(50, 2))
cluster2 = np.random.normal(loc=[50, 60], scale=4, size=(50, 2))
cluster3 = np.random.normal(loc=[80, 40], scale=3, size=(50, 2))

X = np.vstack([cluster1, cluster2, cluster3])
df = pd.DataFrame(X, columns=['Feature_1', 'Feature_2'])
```

Perform the following tasks:

- Scale the features using StandardScaler. Clustering is distance-based, so scaling is essential.
- Use the Elbow Method to find the optimal number of clusters. Run KMeans with k=1 to 10, compute inertia for each, and plot the elbow curve.
- Train a KMeans model with the optimal number of clusters (should be 3). Set random_state=42 and n_init=10.
- Add the cluster labels as a new column Cluster to the dataframe. Print the count of points in each cluster.
- Create a scatter plot of Feature_1 versus Feature_2, colored by cluster label. Mark the cluster centers with a different marker.
- Calculate the silhouette score for the clustering result. What does this score tell you about cluster quality?

Explain in 2-3 sentences: How is the Elbow Method used to choose K? Why does K-Means require feature scaling?

Q5. Scikit-Learn -- PCA (Dimensionality Reduction)

Reduce the number of features while preserving most of the variance. PCA is essential for high-dimensional data visualization.

```
import pandas as pd
import numpy as np
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

np.random.seed(42)
n = 200
df = pd.DataFrame({
    'Height': np.random.normal(170, 10, n),
    'Weight': np.random.normal(70, 15, n),
    'Age': np.random.randint(18, 60, n),
    'Income': np.random.normal(50000, 15000, n),
    'Experience': np.random.randint(1, 20, n),
    'Score_A': np.random.normal(75, 10, n),
    'Score_B': np.random.normal(80, 12, n),
    'Score_C': np.random.normal(70, 8, n)
})
# Add correlation
df['Weight'] = df['Height'] * 0.5 + np.random.normal(0, 5, n)
df['Score_B'] = df['Score_A'] * 0.8 + np.random.normal(0, 5, n)
```

Perform the following tasks:

- Scale all features using StandardScaler. PCA is sensitive to scale, so this step is mandatory.
- Apply PCA to reduce the data to 2 components. Print the explained_variance_ratio_ for each component.
- Calculate the cumulative explained variance. How many components are needed to explain at least 90% of the variance?
- Create a scatter plot of PC1 versus PC2. Color the points by the Age column (use pd.cut to create age groups: Young, Middle, Senior).
- Print the PCA components_ matrix. Identify which original features contribute most to PC1 and PC2.
- Create a bar plot showing the explained variance ratio for each of the 8 principal components.

Explain in 2-3 sentences: Why is PCA useful during EDA? What information do you lose when reducing dimensions, and what do you gain?

Q6. Scikit-Learn -- Decision Tree Classifier

Build an interpretable tree-based model and visualize how decisions are made.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.preprocessing import LabelEncoder

np.random.seed(42)
n = 250
df = pd.DataFrame({
    'Age': np.random.randint(18, 65, n),
    'Income': np.random.normal(50000, 15000, n),
    'Credit_Score': np.random.randint(300, 850, n),
    'Employment': np.random.choice(['Employed', 'Self-Employed', 'Unemployed'], n)
})
# Target: loan default risk
df['Risk'] = np.where(
    (df['Credit_Score'] < 550) | (df['Income'] < 35000) | (df['Employment'] == 'Unemployed'),
    'High', 'Low'
)
```

Perform the following tasks:

- Encode the Employment column using LabelEncoder. Combine it with the numeric columns to create the feature matrix X.
- Encode the Risk target column using LabelEncoder. Split the data into 80-20 train-test with random_state=42.
- Train a DecisionTreeClassifier with max_depth=4 and random_state=42. Print the feature importances.
- Visualize the decision tree using plot_tree(). Display the tree with filled=True, feature_names, and class_names.
- Make predictions on the test set and calculate accuracy. Print the confusion matrix.
- Predict the risk for a new customer: Age=30, Income=40000, Credit_Score=520, Employment=Unemployed. What risk does the model predict?

Explain in 2-3 sentences: Why are decision trees considered interpretable models? What is the risk of setting max_depth too high?

Q7. Scikit-Learn -- Cross-Validation & Model Comparison

Evaluate models robustly using cross-validation and compare multiple algorithms.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import make_scorer, f1_score

np.random.seed(42)
n = 400
df = pd.DataFrame({
    'Feature1': np.random.normal(50, 10, n),
    'Feature2': np.random.normal(30, 8, n),
    'Feature3': np.random.normal(70, 15, n),
    'Feature4': np.random.normal(20, 5, n)
})
df['Target'] = (df['Feature1'] + df['Feature2'] * 0.5 - df['Feature3'] * 0.3 > 40).astype(int)
```

Perform the following tasks:

- Scale the features using StandardScaler. Use the scaled features and Target for all subsequent tasks.
- Perform 5-fold cross-validation on a LogisticRegression model. Print the accuracy for each fold and the mean accuracy.
- Perform 5-fold cross-validation on a DecisionTreeClassifier (max_depth=5). Compare its mean accuracy with LogisticRegression.
- Perform 5-fold cross-validation on a RandomForestClassifier (n_estimators=100). Print the mean and standard deviation of accuracy scores.
- Use StratifiedKFold with 5 splits instead of regular KFold. Explain why stratification matters for classification tasks.
- Create a comparison table showing the mean accuracy and standard deviation for all three models. Which model performs best?

Explain in 2-3 sentences: Why is cross-validation more reliable than a single train-test split? What does the standard deviation of CV scores tell you about model stability?

Q8. Scikit-Learn -- Random Forest & Feature Importance

Build an ensemble model and analyze which features drive predictions.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from sklearn.metrics import accuracy_score, mean_squared_error
from sklearn.preprocessing import StandardScaler

np.random.seed(42)
n = 500
df = pd.DataFrame({
    'Temperature': np.random.normal(25, 5, n),
    'Humidity': np.random.normal(60, 10, n),
    'Wind_Speed': np.random.normal(15, 5, n),
    'Pressure': np.random.normal(1013, 20, n),
    'Cloud_Cover': np.random.uniform(0, 100, n)
})
# Target: will it rain?
df['Rain'] = np.where(
    (df['Humidity'] > 65) & (df['Cloud_Cover'] > 60) & (df['Pressure'] < 1013),
    1, 0
)
```

Perform the following tasks:

- Split the data into 80-20 train-test with `random_state=42`. Scale the features using `StandardScaler`.
- Train a `RandomForestClassifier` with `n_estimators=100`, `max_depth=6`, and `random_state=42`.
- Print the feature importances sorted in descending order. Create a horizontal bar plot showing the importance of each feature.
- Predict on the test set and calculate accuracy. Compare this with a single `DecisionTreeClassifier` trained on the same data.
- Train a `RandomForestRegressor` to predict Humidity from the other features. Calculate the R^2 score and MSE on the test set.
- Experiment with different `n_estimators` values (50, 100, 200) and observe how accuracy changes. Plot accuracy versus `n_estimators`.

Explain in 2-3 sentences: How does a Random Forest reduce overfitting compared to a single Decision Tree? Why is feature importance useful during EDA?

Q9. Scikit-Learn -- Hyperparameter Tuning (GridSearchCV)

Find the best model configuration automatically using systematic hyperparameter search.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

np.random.seed(42)
n = 300
df = pd.DataFrame({
    'Feature1': np.random.normal(0, 1, n),
    'Feature2': np.random.normal(0, 1, n),
    'Feature3': np.random.normal(0, 1, n)
})
df['Target'] = (df['Feature1'] * 2 + df['Feature2'] * -1 + df['Feature3'] * 0.5 > 0).astype
```

Perform the following tasks:

- Split the data into 75-25 train-test with `random_state=42`. Scale the features using `StandardScaler`.
- Define a parameter grid for `SVC` with `C` values `[0.1, 1, 10]`, kernel options `[rbf, linear]`, and gamma `[scale, auto]`.
- Use `GridSearchCV` with 5-fold cross-validation to find the best hyperparameters for `SVC`. Print the best parameters and best cross-validation score.
- Train the best `SVC` model on the full training data and evaluate its accuracy on the test set.
- Repeat the `GridSearchCV` process for `RandomForestClassifier`. Search over `n_estimators` `[50, 100]`, `max_depth` `[3, 5, 7]`, and criterion `[gini, entropy]`.
- Compare the test accuracy of the tuned `SVC` versus the tuned `RandomForest`. Which model performs better on this dataset?

Explain in 2-3 sentences: What is the difference between a hyperparameter and a model parameter? Why is cross-validation used inside `GridSearchCV` instead of a single validation set?

Q10. Scikit-Learn -- Complete ML Pipeline

Build an end-to-end machine learning pipeline that combines preprocessing, model training, and evaluation in a single workflow.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, LabelEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

np.random.seed(42)
n = 400
df = pd.DataFrame({
    'Age': np.random.randint(18, 70, n),
    'Income': np.random.normal(50000, 15000, n),
    'Gender': np.random.choice(['Male', 'Female'], n),
    'City': np.random.choice(['NY', 'LA', 'Chicago', 'Houston'], n),
    'Score': np.random.normal(75, 15, n)
})
# Induce missing values
df.loc[np.random.choice(df.index, 30), 'Age'] = np.nan
df.loc[np.random.choice(df.index, 20), 'Income'] = np.nan
# Target
df['Purchased'] = ((df['Age'] > 30) & (df['Income'] > 45000) & (df['Score'] >
```

Perform the following tasks:

- Identify numeric and categorical columns. Create a ColumnTransformer that applies StandardScaler to numeric columns and OneHotEncoder to categorical columns.
- Add SimpleImputer with strategy median for numeric columns and most_frequent for categorical columns inside the ColumnTransformer.
- Build a Pipeline that chains the preprocessor with a RandomForestClassifier. Train the pipeline on the training data.
- Build a second Pipeline with LogisticRegression. Compare the test accuracy of both pipelines.
- Use cross_val_score with 5 folds directly on the pipeline (not on separate steps). Print the mean CV accuracy for both pipelines.
- Predict the Purchase outcome for a new customer: Age=35, Income=55000, Gender=Female, City=LA, Score=80. Use the best pipeline.

Explain in 2-3 sentences: Why is using a Pipeline better than manually preprocessing data before modeling? What errors can occur if preprocessing is fit on the entire dataset instead of just the training set?